ZoikoShield

Complete Frontend, Backend & Email Guide

Version 3 - page by page, step by step, with the reason for every choice

Part A Architecture, ports and why the platform is split this way

Part B Frontend foundation: shell, data layer, UI states, design

Part C Every frontend page: flow, logic, backend call and WHY

Part D The email system in full (templates, gates, every email path)

Part E Backend services explained step by step

Part F End-to-end ERB-01 flow, UI -> API -> service

Part G Setup and troubleshooting runbook (incl. 'Invalid credentials')

Part H Verified findings and remaining work

Source of truth: the repository code (frontend/src, backend/apps). Generated by backend/scripts/generate-detailed-platform-guide-pdf.

How to read this guide

This guide was written so that a new engineer, a product owner, an auditor or a customer success person can each understand the platform without asking anyone. Every page of the product is described the same way: **what the user sees**, **the steps**, **how the frontend works**, **which backend service answers**, and **why** it was built that way. Where the real code has a gap or a mismatch, it is stated plainly in a red box instead of being hidden.

What ZoikoShield is, in one paragraph

ZoikoShield is a multi-tenant cloud security platform. Customer systems (GitHub, Azure Monitor, cloud logs, webhooks) send security events in. The platform **normalizes** them, runs **detections**, raises **alerts**, lets analysts open **cases**, lets an **AI copilot** summarise and suggest, lets approved people run **SOAR response actions**, and finally proves everything with a **tamper-evident Merkle evidence ledger** that an auditor can verify offline. Every step is tenant-isolated, role-checked, and audited.

Contents

Part	Topic	Approx. pages
A	Architecture, ports, why a proxy, why separate services	3-5
B	Frontend foundation: shell, data layer, 7 UI states, design system	6-9
C	Page-by-page frontend guide (25 page cards)	10-37
D	Email system: templates, gates, every email path, status	38-44
E	Backend services step by step	45-53
F	End-to-end ERB-01 flow	54-55
G	Setup and troubleshooting runbook	56-57
H	Verified findings and remaining work	58-60

Verification basis

- Every page and service in this guide was read from the repository source (frontend/src/app, frontend/src/lib, frontend/src/components, backend/apps/*).
- The page -> API wiring map in Part C was built from the real calls in the page files and lib/api-client.ts.
- Older documents under docs/ disagree with the code in a few places (ports, password hashing, demo fallback). The code wins; see Part H.
- Page numbers in the contents are approximate because section lengths can change when the generator is re-run.

Part A

Architecture, ports and why it is split this way

Before looking at pages, understand the shape of the system. The browser never talks to the security services directly. It talks to one Next.js app, which forwards to the right backend service.

A1. The services and their ports

Service	Port	Responsibility	Why separate
frontend (Next.js)	3000	UI, plus a server-side proxy at /api/v1/[...slug]	One origin for the browser; secrets stay on the server
shield-core	3001	Identity, tenants, RBAC, auth, onboarding, cases, notifications, audit, governance	Everything that must be consistent and transactional
shield-ingest	3002	Connectors, webhooks, normalization, events	Highest traffic; must scale and fail alone
shield-ai	3003	Copilot, grounding gate, model-armor safety gateway, AI governance	AI is risky; isolating it limits blast radius
shield-action	3004	SOAR actions, freeze, rollback, dual-custody quorum	Actions change customer systems; strict control
shield-anchor	3005	Merkle tree, anchoring, receipts, proofs	Evidence integrity must be independent of the app
verifier-cli	offline	Re-verifies Merkle proofs without the platform	Auditors must not have to trust our servers
PostgreSQL / Redis / Redpanda	5432 / 6379 / 9092	Data / cache and rate limits / Kafka events	Durable state, speed, async decoupling

Older documents list ports 3000-3004. The code (`shield-anchor` on 3005 and `shield-core` reading `PORT`) is authoritative.

A2. How one click travels through the system

1. The user clicks a button in a page component (React, in the browser).
2. The page calls a method on `lib/api-client.ts` (`ZoikoShieldApiClient`) or `lib/backend.ts` (`backend.get/post/patch`).
3. That sends `fetch('/api/v1/...')` to the **same origin** (Next.js on port 3000) and attaches `x-tenant-id`.
4. The Next.js route `frontend/src/app/api/v1/[...slug]/route.ts` decides which backend service owns that path and forwards the request with the Authorization header and cookies.
5. The backend service authenticates, authorizes (RBAC, tenant check, Cedar-style policy), does the work, writes an audit record and returns JSON.
6. The proxy returns the real response (or a real error) to the browser; the page shows data or one of the 7 mandatory UI states.

A3. Why a proxy instead of calling services directly

WHY

- **One origin = no CORS pain and cookies work.** The session cookie is set for the Next.js origin; browsers never need to know five backend hosts.
- **Secrets stay on the server.** Webhook calls under `ingestion/webhooks/*` are signed server-side with HMAC `${timestamp}.${nonce}.${body}` using `WEBHOOK_HMAC_SECRET`; the key is never shipped to the browser.
- **One place to route.** `ingestion/*` and `events*` go to ingest, `ai/*` and `copilot*` to AI, `actions/*` and `response/*` to action, `anchor/*` and `ledger/*` to anchor, everything else to core.
- **Honest errors.** The proxy no longer fabricates data when a service is down: it returns `502 Bad Gateway` (15 s timeout, `BACKEND_PROXY_TIMEOUT_MS`) so the UI shows a real degraded state.
- **Streaming works.** Server-sent events are passed straight through for live pages.

A4. Proxy routing table

Path prefix	Goes to	Env variable
ingestion/*, events*	shield-ingest :3002	SHIELD_INGEST_URL
ai/*, copilot*	shield-ai :3003	SHIELD_AI_URL
actions/*, response/*	shield-action (or core)	SHIELD_ACTION_URL
anchor/*, ledger/*	shield-anchor (or core)	SHIELD_ANCHOR_URL
everything else (auth, tenants, cases, alerts, notifications...)	shield-core :3001	SHIELD_CORE_URL

A5. Principles used everywhere

- **Tenant isolation first:** every request carries a tenant; resources from another tenant are rejected, not hidden.
- **Least privilege:** actions need explicit permissions such as `tenant:member:invite`; risky actions need two people.
- **Fail closed:** unknown template, bad signature, missing field, secret in an email - all are refused, never best-effort sent.
- **Everything is audited and provable:** audit rows feed the Merkle ledger so evidence cannot be silently edited.
- **Honest UI:** loading, stale, partial, degraded, unauthorized, unavailable and recovery are separate visible states.

Part B

Frontend foundation

Pages are thin. The shell, the data layer and the shared UI states do most of the work. Understanding these four pieces makes every page card in Part C easy to read.

B1. The application shell (what is on every page)

- **TrustBar (top):** tenant organisation name, environment badge (for example PRODUCTION-US-EAST), region (for example eu-west-1), 'Evidence: Signed Merkle checkpoints', 'Microservices: 5/5 OK', and the signed-in user's name and role (for example ANALYST OPS / SECURITY_ANALYST).
- **Sidebar (left):** grouped navigation with small status badges, plus a footer showing 'Merkle Epoch #1043' and 'PQC Seal ML-DSA-65'.
- **DemoGuideBar (bottom):** the ERB-01 runbook stepper, stages 1-10, with Prev / Next Stage. It walks a demo audience through the product in order.
- **Content area:** the active page.

WHY

- The TrustBar answers 'which customer, which environment, which region, who am I?' on every screen. In security tooling, acting in the wrong tenant is the worst mistake, so it is always visible.
- Sidebar groups mirror the real job of a SOC team: investigate, buy/configure, connect and comply, administer, govern the platform.
- The DemoGuideBar lets sales, QA and auditors replay one fixed story (ERB-01) the same way every time.

Note: 'Microservices: 5/5 OK', 'Merkle Epoch #1043' and 'PQC Seal ML-DSA-65' are currently static labels. Real health is on the Platform Service Health page.

B2. Sidebar groups and every entry

Group	Entries (route)
SEC-OPS & INVESTIGATION	Command Center (/), AI Security Copilot (/copilot, 6 MODES), Ingestion Console (/ingestion), Detection & Alerts (/alerts), Case Workspace (/cases), AI Safety & Incidents (/ai-governance), Threat Hunting Copilot (/hunting, ReAct), Playbook Runs W18 (/playbooks), Response Proposals W34, Simulation Replay (/red-team), SOAR Actions & Freeze (/actions, R0-R4), IR Retainer (/ir-retainer, 24x7)
COMMERCIAL & CATALOGUE	Services Catalogue (/services), Plans & Band Pricing (/pricing), Sector Solution Packs (/sector-packs)
CONNECTIVITY & COMPLIANCE	Connectors Wizard (/connectors), Policy Lifecycle W12 (/policies, 4-EYES), Merkle Evidence Ledger (/ledger, ZS-MERKLE), Controls & Compliance (/controls), Evidence Operations W24, Risk & Exceptions W27/28 (/risk), Notification Center W04 (/notifications), Audit & Verifier (/audit, SEALED)
TENANT ADMINISTRATION	Team & Invitations (/team), Organization Onboarding (/onboarding), MSSP Fleet & 2-Party JIT (/partner/fleet), Switch Account / Login (/login)
PLATFORM ADMIN & GOVERNANCE	Platform Service Health (section 31/32), G1 Launch Gate Protocol (0/8 GATE), JIT Elevation Quorum (section 13, FOUR-EYES), GTM Pre-Flight Checklist (12/12 PASS), Operations & Entitlements (/admin, OPERATOR)

Group	Entries (route)
G2 EXPERIENCE CONTRACTS	Incident Command W19, SOC Shift Handover W20, Asset Inventory W29, Findings W30, Executive Risk W31, Audit Reports W32, Admin Export W36, Developer W37, Trust W38

B3. The data layer (how pages get data)

- `lib/api-client.ts` (`ZoikoShieldApiClient`, about 5,600 lines): one typed method per backend operation. All pages use it, so URLs and payload shapes live in one file.
- `safeFetch(fn, fallbackFn)`**:** runs the real call. A demo fallback is used **only if** `NEXT_PUBLIC_DEMO_FALLBACK === 'true'`. By default it is off, so a failure surfaces as a real error. (Older documents that claim 100% demo fallback are outdated.)
- `lib/backend.ts`**:** a smaller helper, `backend.get/post/patch`, throws `BackendError`, and `asList()` unwraps `items | results | records | data` arrays.
- `lib/demo-state.ts`**,** `contract-state.ts`**:** remember the active tenant id, user session and BFF contract state in the browser.
- `lib/webauthn.ts`**:** wraps the browser passkey (FIDO2) APIs.
- `lib/use-event-stream.ts`**:** subscribes to server-sent events for live updates.
- `lib/types.ts`**:** shared TypeScript types that mirror backend DTOs.

WHY

- A single client means one place to add the tenant header, error handling and retries. A page cannot forget the tenant.
- Fallback is opt-in because silent fake data is dangerous in a security product: an analyst could believe an alert list is real when the backend is down.
- Typed methods give compile-time protection when a DTO changes (although see Part H: a few DTO shapes still drift).

B4. The seven mandatory UI states

Defined in `components/states/mandatory-ui-states.tsx`; each is announced with `aria-live` so screen readers hear changes.

State	Meaning	Typical trigger
LOADING	Request in progress	Page just opened
PARTIAL	Some data loaded, some missing	One of several service calls failed
STALE	Data shown is older than its freshness window	Live feed lagging
DEGRADED	System running on a safe fallback	AI kill switch active; deterministic rule engine used
UNAUTHORIZED	Caller lacks permission or step-up	HTTP 401 / 403
UNAVAILABLE	Service not reachable	Proxy 502 / timeout
RECOVERY	Service came back; refreshing	After retry succeeds

B5. UI kit and design language

- `components/ui`: Card, Badge, Button, Modal, Tabs. Pages compose these instead of inventing styles, so the product looks consistent.
- Dark theme with cyan/blue accents, status colours (green pass, amber warn, red fail, purple governance) and short badges such as 4-EYES, SEALED, R0-R4.
- Badges are used to put **control information next to the name** (for example '4-EYES' on Policy Lifecycle) so users learn the rule before they click.

WHY

- A dark, dense layout is the convention for SOC tools used for long shifts; contrast badges let an analyst scan state in a second.

- Shared components mean accessibility fixes (focus, aria) are made once and apply everywhere.

Part C

Page-by-page frontend guide

25 page cards follow. Each describes the screen, the steps, the logic, the backend call and the reason. Order follows a real customer journey: sign in, onboard, see the command center, connect data, detect, investigate, respond, prove, administer.

C1. Login / Switch Account

Route: /login | Source: frontend/src/app/login/page.tsx

What the user sees

- Fields: **Analyst Identity Email**, **Password Fallback Credential**, **Tenant ID** (with the hint 'npm run seed:dev-tenant prints one').
- Preset buttons (sarah.chen@acme.com, owner@acme.com, admin@zoikoshield.io) for quick demo sign-in.
- Primary button 'Authenticate & Issue JWT Bearer Token' and passkey (FIDO2/WebAuthn) options.
- On failure: a red banner with the message and, above it, a red policy line.

Step-by-step flow (what you do, what happens)

1. Open /login. Email defaults to `analyst@zoikoshield-demo.com`; tenant id comes from saved demo state.
2. Type email, password and tenant id (a UUID such as `00000000-0000-4000-8000-000000000001`).
3. Click Authenticate. The page calls `POST /api/v1/auth/login` with `x-tenant-id`.
4. Success: the backend sets JWT and refresh cookies; the page stores the session (name, role, tenant) and sends you to the Command Center.
5. Failure: the page shows the error and the UnauthorizedState. Passkey path: `/auth/passkeys/authentication/options` then `/auth/passkeys/authentication`.

Frontend logic (how the page is built)

- Any 401/403, or any message containing 'Cedar' or 'FIDO2', renders UnauthorizedState with a **hard-coded label**: 'CEDAR_POLICY_DENY: Authentication step-up required'.
- That red policy line is **not a real Cedar decision**. It is a generic label chosen by the page. In your screenshot the real cause is the line below it: backend `Invalid credentials`.
- Tenant id is a required field because the backend binds a session to exactly one tenant.

Backend call and what the backend does

- `AuthService.login` looks up the principal, checks status, loads the local credential and compares the password with **bcrypt**.
- Lockout after `LOCKOUT_THRESHOLD` failed attempts. Unverified email returns 403 'Email not verified'. Throttle 10 requests per minute.
- `sessionContext.resolveBinding` confirms the user belongs to the tenant, then JWT + refresh tokens are issued in cookies and identity events are written.
- Unknown email, inactive user, no local credential and wrong password all return the **same** `Invalid credentials` (anti-enumeration).
- Passkey login produces a PASSKEY assurance level for step-up decisions.

WHY we built it this way

- Anti-enumeration: an attacker must not learn which emails exist, so all failures look identical.
- Passwords are a fallback; passkeys give phishing-resistant assurance, which policies can require for sensitive actions.
- Cookies (HttpOnly) rather than tokens in JavaScript protect sessions from script theft.
- Tenant binding at login prevents a valid user from silently acting in the wrong customer.

Verified gaps on this page (see Part H)

- Your 'Invalid credentials' means that email/password/tenant combination does not exist in the database. Fix: see Part G (seed commands).
- Hard-coded 'CEDAR_POLICY_DENY' label is misleading and should show the backend message only.

- Older docs say Argon2id; the code uses bcrypt.

C2. Organization Onboarding

Route: /onboarding | Source: frontend/src/app/onboarding/page.tsx

What the user sees

- A guided form to create a new customer tenant: organisation name, slug, legal entity, environment name, home region, data class, order id.
- A 'Seed New Commercial Order' helper button that fills an order id.

Step-by-step flow (what you do, what happens)

1. Fill organisation name and slug (URL-safe short name).
2. Choose the home region (data residency) and environment.
3. Submit. The page calls `POST /api/v1/onboarding`.
4. On success the backend creates the tenant, the owner principal and an owner activation invitation, and the owner receives an activation email (ZS-EML-ORG-002).
5. The owner opens the link, sets credentials and signs in.

Frontend logic (how the page is built)

- Sends `{organizationName, slug, legalEntityName, environmentName, homeRegion}`. `orderId` and `dataClass` are collected but **not sent**.
- 'Seed New Commercial Order' only generates a random string; it does not call a billing system.
- Uses the shared Button/Card components and shows loading and error states.

Backend call and what the backend does

- `OnboardingService` creates tenant, legal entity, owner principal and invitation in one flow and emits audit events.
- `MailService.sendOwnerInvitation` renders template `ZS-EML-ORG-002`; link is `${APP_BASE_URL}/accept-invite?token=...`.
- Backend DTO `OnboardTenantDto` (whitelist on) requires `orderId, tenantName, tenantSlug, homeRegion, legalEntity{...}, ownerEmail, accessDisclosureVersion`.

WHY we built it this way

- Tenant creation is the legal and commercial start of a customer: region fixes where data lives; legal entity drives contracts and privacy.
- Ownership starts with an invitation (not a password we invent) so the owner proves control of the email address.
- Order id ties provisioning to a purchase so that tenants cannot appear without a commercial record.

Verified gaps on this page (see Part H)

- **Contract mismatch:** the frontend payload does not match `OnboardTenantDto` (field names, missing ownerEmail/orderId/legalEntity). Real onboarding will return 400 until aligned.
- The activation link points to `/accept-invite`, but **no such frontend page exists** yet.

C3. Command Center (home)

Route: / | Source: frontend/src/app/page.tsx

What the user sees

- Top banner 'BFF EXPERIENCE CONTRACT (LAB 14): HEALTHY_SYNCED' with freshness and correlation id.
- Hero 'ZoikoShield SecOps & Cryptographic Command Center' with buttons 'AI Safety Cockpit' and 'Simulate Telemetry'.
- Amber 'AI Safety Guardrail Degraded' notice when a model is under containment.
- Four metric cards: Unresolved Alerts, AI Safety & Drift, Active Connectors, Compliance Posture.
- ERB-01 Demonstration Runbook: 8 cards covering steps 1-12 across the services.

Step-by-step flow (what you do, what happens)

1. Land on the page after login.
2. Read the banner: is the experience contract healthy and how fresh is the data?
3. Look at the four cards to decide where to go first (alerts, AI incidents, connectors, compliance).
4. Click a runbook card, or use the bottom stepper (stages 1-10) to follow the story.
5. Use 'Simulate Telemetry' to push demo events through the pipeline.

Frontend logic (how the page is built)

- Calls several client methods in parallel (alerts, cases, connectors via `getConnectors`, control tests).
- Conditionally renders Loading, Stale, Partial or Degraded states from the shared state components.
- Compliance posture is computed from control test results; connector count and ingest health come from the live connector list.

Backend call and what the backend does

- Reads: alerts, cases, connectors (shield-ingest), control tests, AI governance status. Each goes through the proxy to the owning service.
- Degraded notice reflects the AI kill switch / `SafeDegradationService` (deterministic rule engine takes over).

WHY we built it this way

- A home page must answer 'is anything on fire and is the platform itself healthy?' in five seconds.
- Showing degradation explicitly (instead of hiding it) is a trust feature: analysts know when AI help is reduced.
- The runbook doubles as built-in onboarding and a demo script.

Verified gaps on this page (see Part H)

- Static TrustBar labels (5/5 OK) may disagree with real health; use Platform Service Health for truth.

C4. Team & Invitations

Route: /team | Source: frontend/src/app/team/page.tsx

What the user sees

- Member list for the tenant with roles and status.
- An invite form (email + role) and a pending invitation list.
- Passkey management: list registered passkeys, register new, revoke.
- A demo button 'Switch to Invitee & Accept' to simulate the invitee side.

Step-by-step flow (what you do, what happens)

1. Open Team. The page loads members from `/tenants/:id/members` and invitations from `/tenants/:id/invitations`.
2. Enter an email and choose a role, click Invite.
3. The backend creates a pending invitation and returns a one-time token.
4. The invitee signs in as that email and accepts via `/invitations/:token/accept`.
5. A new TenantMembership appears in the list.

Frontend logic (how the page is built)

- Uses `backend` helper and API client methods; refreshes lists after each action.
- Passkey section uses `lib/webauthn.ts` (`/auth/passkeys/registration/options`, `/registration`, list and revoke).
- Errors are currently only written to the console.

Backend call and what the backend does

- Requires permission `tenant:member:invite`. Platform-level roles cannot be granted by invitation.
- Token = 32 random bytes in hex; only its **SHA-256 hash** is stored. Default expiry applies.
- Accept requires an authenticated principal whose email equals the invited email, status PENDING and not expired; membership source is INVITATION.
- **No email is sent** for tenant member invitations: the token is shown in the UI.

WHY we built it this way

- Invitations are the only safe way to add people: the account owner (not an admin typing a password) proves identity.
- Storing a hash means a database leak cannot be replayed to join a tenant.
- Email-match on accept stops a leaked link being used by a different person.

Verified gaps on this page (see Part H)

- Frontend `inviteAnalyst` sends `{invitedEmail, assignedRole}` but backend `CreateInvitationDto` needs `{invitedEmail, roleId}`; `assignedRole` is stripped by the whitelist, so real invites likely fail with 400 and the page does not tell the user.
- Backend returns `{invitationId, expiresAt, token}`, while the page expects an Invitation object. Accept returns a membership, but `acceptInvitation` reads `inv.invitedEmail`, which would break.
- 'Switch to Invitee' signs in with the demo password `demo-password`.

C5. Connectors Wizard

Route: /connectors | Source: frontend/src/app/connectors/page.tsx (+ components/connectors/ConnectorConfigModal.tsx)

What the user sees

- A catalogue of connector types (for example GitHub, Azure Monitor, cloud logs, generic webhook) with a count badge in the sidebar.
- A configuration modal per connector with credentials, scopes and options.
- Status per connector: configured, active, testing, error, last sync.

Step-by-step flow (what you do, what happens)

1. Pick a connector card and click Configure.
2. Fill the modal fields (the form depends on the connector type) and Save.
3. Click **Test** to verify credentials and reachability.
4. Click **Activate**; the backend starts ingesting (webhook endpoint or poller).
5. Click **Sync** to pull now; watch status and last-sync time update.

Frontend logic (how the page is built)

- Calls `/connectors` (list/create/update) plus actions `activate`, `test`, `sync`.
- `ConnectorConfigModal` renders typed fields and validates before sending; secrets are write-only fields.
- Active connector count feeds the Command Center card.

Backend call and what the backend does

- Connector catalogue and workers live in shield-ingest (for example `azure-monitor-poller.service.ts`).
- Secrets are stored encrypted; the UI never receives them back.
- Webhook connectors get a signed endpoint `/ingestion/webhooks/:connectorId` (see Ingestion).

WHY we built it this way

- Data is the fuel of detection; a wizard reduces misconfiguration, the leading cause of silent blind spots.
- Test-before-activate catches bad credentials before analysts rely on a feed.
- Write-only secrets mean screen sharing or support tickets cannot leak keys.

C6. Ingestion Console

Route: /ingestion | Source: frontend/src/app/ingestion/page.tsx

What the user sees

- Live view of incoming events, per-connector throughput and health, and a way to send test telemetry.
- A badge in the sidebar with the number of active streams.

Step-by-step flow (what you do, what happens)

1. Open the console; the page loads `/events` and connector health.
2. Optionally subscribe to the live stream (server-sent events).
3. Click 'Simulate Telemetry' (from Command Center) or send a test webhook.
4. Watch normalized events appear with source, severity and timestamp.

Frontend logic (how the page is built)

- `use-event-stream.ts` opens an SSE connection through the proxy and appends events.
- Webhook tests go to `/ingestion/webhooks/:connectorId`; the **proxy adds the HMAC signature** server-side.
- Stale and Degraded states appear if the stream stops.

Backend call and what the backend does

- `WebhookSignatureGuard` requires the raw body, verifies HMAC-SHA256 over `timestamp.nonce.body`, enforces a +/-300 second window and `timingSafeEqual`.
- A replay nonce is stored as a SHA-256 hash; a repeat returns 401 'already processed'. GitHub uses `X-Hub-Signature-256` plus `X-GitHub-Delivery` (24 h replay key).
- Per-connector secrets come from `WEBHOOK_HMAC_SECRETS`; the global `WEBHOOK_HMAC_SECRET` works only outside production. The tenant is bound from the connector.
- Events are then normalized (protected baseline) into a common schema and published to Kafka.

WHY we built it this way

- Signatures + timestamp + nonce stop forged, delayed and replayed events, which would otherwise poison detections.
- Server-side signing keeps the shared secret out of browsers.
- A live console lets operators see a broken feed within seconds, not days.

C7. Detection & Alerts

Route: /alerts and /detections | Source: frontend/src/app/alerts/page.tsx, frontend/src/app/detections/page.tsx

What the user sees

- Alerts list with severity (P1 critical, etc.), source, time and status; filters.
- Detections page: rule definitions, recent matches and version history with a Publish action.
- Button 'Create Case' on an alert.

Step-by-step flow (what you do, what happens)

1. Open Alerts; pick an alert to see context.
2. Click **Create Case** (`POST /alerts/:id/create-case`); you land in the Case Workspace.
3. On Detections: browse `/detections/definitions`, `/detections/matches`, and publish a new version via `/detections/versions/:id/publish`.
4. Published versions apply to new events.

Frontend logic (how the page is built)

- List components with filters and badges; actions disable while pending; errors use shared states.
- Publish confirmation prevents accidental rule changes.

Backend call and what the backend does

- Detection rules run on normalized events; a match creates an alert (threshold rules can generate a P1).
- Alert-to-case creation copies evidence references and writes an audit record.
- Rule versions are immutable once published, so each alert points to the exact rule that fired.

WHY we built it this way

- Separating definitions, matches and alerts lets teams tune noisy rules without losing history.
- Versioned publish = you can always explain why an alert fired months later (audit requirement).
- One-click case creation shortens time-to-triage.

C8. Case Workspace

Route: /cases (and /cases/incidents/command) | Source: frontend/src/app/cases/*

What the user sees

- Case list and detail: timeline, linked alerts, evidence, assignee, status and an AI summary panel.
- Incident Command (W19) view for major incidents, and SOC Shift Handover (W20).

Step-by-step flow (what you do, what happens)

1. Open a case from the sidebar or from an alert.
2. Review the timeline and evidence.
3. Click **AI summary** (`/cases/:id/ai/summary`) to get a grounded summary.
4. Propose or request a response action (goes through approval; see Playbooks and SOAR).
5. Update status, add notes; hand over at shift end.

Frontend logic (how the page is built)

- Reads `/cases` and `/cases/:id`; AI summary call is explicit (user triggered).
- Shows grounding citations so a human can check the AI claim.

Backend call and what the backend does

- Cases live in shield-core (transactional with audit). The summary request goes to shield-ai, where `GroundingGateGuard` requires the answer to be backed by case evidence.
- Model Armor screens inputs and outputs for prompt injection and data leakage.

WHY we built it this way

- Cases are the unit of accountability: who looked, what they decided, and why.
- AI summary is optional and cited because analysts, not the model, own the decision.
- Handover pages exist because shift changes are where incidents are dropped.

C9. AI Security Copilot

Route: /copilot | Source: frontend/src/app/copilot/page.tsx

What the user sees

- A chat-style assistant with six modes (shown as '6 MODES' in the sidebar).
- Answers include citations and a decision card (approve / reject) for proposed actions.

Step-by-step flow (what you do, what happens)

1. Choose a mode and type a question about alerts, cases or posture.
2. The page posts to `/copilot/query`; the answer streams or arrives with citations.
3. If the AI proposes an action, you see an envelope with Approve / Reject.
4. Your choice calls `/ai/decisions/:envelopeId/:decision`.

Frontend logic (how the page is built)

- Chat state is local; each message keeps its citations and guard verdict.
- Degraded state is shown when the AI kill switch is active.

Backend call and what the backend does

- shield-ai: Model Armor safety gateway on input/output, `GroundingGateGuard` to block ungrounded claims, tenant-scoped retrieval only.
- The AI **cannot directly trigger live SOAR**; it can only create proposals that humans must approve.

WHY we built it this way

- LLMs can hallucinate and be manipulated. Grounding + safety gateway + human approval give useful speed without handing the model the keys.
- Modes narrow the task, which reduces risky free-form output.

C10. Threat Hunting Copilot

Route: /hunting | Source: frontend/src/app/hunting/page.tsx

What the user sees

- A ReAct-style investigation view: the assistant reasons in steps (thought, action, observation) and builds an attack path for a case.

Step-by-step flow (what you do, what happens)

1. Select a case.
2. Start the hunt; steps appear one by one.
3. Inspect the attack path graph (`/ai/cases/:id/attack-path`).
4. Save findings to the case.

Frontend logic (how the page is built)

- Renders each reasoning step so the analyst can follow and challenge it.
- Read-only queries; no actions are executed here.

Backend call and what the backend does

- shield-ai builds the path using only tenant data and returns structured steps with sources.
- Same guard chain as Copilot.

WHY we built it this way

- Showing the reasoning (not just the answer) builds trust and lets junior analysts learn.
- Read-only by design: hunting should never change the customer environment.

C11. AI Safety & Governance

Route: /ai-governance | Source: frontend/src/app/ai-governance/page.tsx

What the user sees

- List of AI incidents (blocked outputs, injection attempts), drift indicators, and the kill-switch / fallback controls.
- Sidebar badge shows number of active incidents.

Step-by-step flow (what you do, what happens)

1. Open the page; incidents load from `/ai/incidents` and drift from `/ai/drift`.
2. Open an incident to see what was blocked and why.
3. An authorised person triggers a fallback with `/ai/incidents/:id/fallback`.
4. The Command Center then shows the Degraded banner and the system uses deterministic rules.

Frontend logic (how the page is built)

- Severity-sorted incident list, drift charts, explicit confirmation for fallback.
- Uses the Degraded state to reflect the current safety posture.

Backend call and what the backend does

- `SafeDegradationService` switches from model output to a strict rule engine when the kill switch is active.
- Events such as `ai.output_blocked` also feed notifications.

WHY we built it this way

- AI governance is a legal and trust requirement; a visible kill switch shows customers they stay in control.
- Falling back to deterministic rules keeps protection running when the model is untrusted.

C12. Playbook Runs and Response Proposals

Route: /playbooks, /response-proposals | Source: frontend/src/app/playbooks/page.tsx, frontend/src/app/response-proposals/page.tsx

What the user sees

- Playbook runs (W18): list of automated response sequences with step status and freeze status.
- Response proposals (W34): AI or analyst suggested actions with simulate, approve or reject buttons.

Step-by-step flow (what you do, what happens)

1. Open Playbooks; runs load from `/playbooks/runs`; freeze state from `/response/freeze-status`.
2. On Proposals pick one; click **Simulate** (`/response-proposals/:id/simulate`) to see predicted impact.
3. Approve or reject (`/response-proposals/:id/:action`).
4. Approved actions move to the action service for execution.

Frontend logic (how the page is built)

- Shows simulation output before enabling Approve, nudging safe behaviour.
- Case links come from `/cases`.

Backend call and what the backend does

- shield-core stores proposals and approvals; shield-action executes. Approver cannot be the proposer for sensitive classes.
- Everything is audited.

WHY we built it this way

- Simulation first avoids 'blast radius' mistakes such as disabling a production account.
- Proposal -> approval -> execution separates thinking from doing.

C13. SOAR Actions and Freeze

Route: /actions | Source: frontend/src/app/actions/page.tsx

What the user sees

- Action catalogue graded R0-R4 by risk, freeze / unfreeze controls and a rollback button.
- Dual-custody approvals for high-risk actions.

Step-by-step flow (what you do, what happens)

1. Choose an action; high-risk (R2-R4) shows that two signers are required.
2. Initiator signs with a passkey (FIDO2) and the request becomes PENDING_SECOND_SIGNATURE.
3. A second authorised person approves within the TTL (default 15 minutes).
4. Status becomes QUORUM_REACHED and the action runs; otherwise REJECTED or EXPIRED.
5. Rollback uses a single-use token and a compensating plan (`/actions/rollback`).

Frontend logic (how the page is built)

- Calls `/response/freeze`, `/response/unfreeze`, `/actions/rollback`.
- Uses WebAuthn for signing; badges R0-R4 explain risk before clicking.

Backend call and what the backend does

- `DualCustodyQuorumService` (ADR-04) enforces the quorum, signature and expiry.
- Executors integrate Cloud Armor, GCP IAM and Google Workspace.
- Rollback tokens are single use so a stolen token cannot be replayed.

WHY we built it this way

- Two-person control is the standard for destructive operations (no single insider or compromised account can act alone).
- Risk tiers keep low-risk work fast and high-risk work safe.

Verified gaps on this page (see Part H)

- Quorum records are held in an **in-memory Map** (lost on restart) and the signing key has a **hard-coded default fallback**; both must be replaced before production.

C14. Simulation Replay and IR Retainer

Route: /red-team, /ir-retainer | Source: frontend/src/app/red-team/page.tsx, frontend/src/app/ir-retainer/page.tsx

What the user sees

- Simulation Replay: replay recorded attack scenarios to test detections and playbooks safely.
- IR Retainer: the customer's incident response retainer, hours, SLA and contact (24x7).

Step-by-step flow (what you do, what happens)

1. Pick a scenario and run the replay; observe alerts, cases and proposals appear.
2. Open IR Retainer to see the contract (`/ir/retainers`), SLA and request escalation.

Frontend logic (how the page is built)

- Replay uses synthetic events; results are labelled so they never mix with real incidents.
- Retainer is a read/request view.

Backend call and what the backend does

- Replay feeds the normal pipeline using a simulation flag; retainer data comes from shield-core.

WHY we built it this way

- Teams must test response before a real incident; simulations are the safe rehearsal.
- A retainer view tells customers exactly what help they are entitled to.

C15. Commercial: Services, Pricing, Sector Packs

Route: /services, /pricing, /sector-packs | Source: frontend/src/app/services, pricing, sector-packs

What the user sees

- Services Catalogue: public capabilities with a count badge.
- Plans & Band Pricing: tiers and bands.
- Sector Solution Packs: pre-built bundles for industries such as finance.

Step-by-step flow (what you do, what happens)

1. Browse services (`/commercial/capabilities/public-services`).
2. Compare plans (`/commercial/plans`).
3. Pick a sector pack to see which connectors, detections, controls and reports it enables.

Frontend logic (how the page is built)

- Read-only pages with cards and comparison tables.

Backend call and what the backend does

- shield-core commercial module serves plans and capabilities; entitlements later gate features per tenant.

WHY we built it this way

- The product and the commercial model must agree; plans drive entitlements so customers only see what they bought.
- Sector packs speed time-to-value for regulated industries.

C16. Policy Lifecycle

Route: /policies | Source: frontend/src/app/policies/page.tsx

What the user sees

- Policy list with states (draft, review, approved, active, retired) and the badge '4-EYES'.

Step-by-step flow (what you do, what happens)

1. Author a draft policy.
2. Submit for review.
3. A **different** person approves (four-eyes).
4. Activate; the platform enforces the new version and records it.

Frontend logic (how the page is built)

- State machine UI: only valid next buttons are enabled.
- Shows version and approver.

Backend call and what the backend does

- Policy versions are immutable; approval requires a second principal; changes are audited and anchored in the ledger.

WHY we built it this way

- Policies (Cedar-style authorization and security policy) change who can do what; they need the same control as code review.
- Immutable versions allow auditors to prove which policy applied at any time.

C17. Merkle Evidence Ledger

Route: /ledger | Source: frontend/src/app/ledger/page.tsx

What the user sees

- Epoch list, receipts, leaf proofs and a 'verify proof' tool; badge ZS-MERKLE.

Step-by-step flow (what you do, what happens)

1. Choose an epoch; the page loads `/anchor/receipts/:epoch`.
2. Request a proof for a leaf (`/anchor/proofs/:epoch/:leaf`).
3. Verify (`/anchor/proofs/verify`).
4. An operator may seal a batch (`/anchor/batches/seal`).

Frontend logic (how the page is built)

- Displays hashes in monospace with copy; shows verification result clearly (valid / tampered).

Backend call and what the backend does

- `MerkleTreeService` (shield-anchor): profile `ZS-MERKLE-V1`; leaf = SHA256(0x00 || bytes); branch = SHA256(0x01 || left || right); odd node is duplicated; proof = list of {siblingHash, position}; `verifyInclusion` recomputes the root.
- Domain-separation bytes (0x00/0x01) prevent second-preimage attacks.

WHY we built it this way

- A ledger turns 'trust us' into 'verify it yourself': any change to a record changes the root.
- Batching into epochs keeps cost low while still giving per-record proofs.

C18. Audit & Verifier, Auditor Workspace, Verify Certificate

Route: /audit, /auditor, /verify-certificate | Source: frontend/src/app/audit, auditor, verify-certificate

What the user sees

- Audit & Verifier: audit packages (badge SEALED), generate and download.
- Auditor workspace: read-only evidence chain for an external auditor.
- Verify Certificate: paste or upload a certificate to check it.

Step-by-step flow (what you do, what happens)

1. Generate an audit package (`/audit-packages`); it is frozen and sealed.
2. Auditor opens `/auditor/workspace` and follows `/auditor/evidence-chain/...`.
3. Run the offline `verifier-cli` on the downloaded package: exit code 0 = valid, non-zero = tampering.
4. Verify Certificate confirms a sealed report is genuine.

Frontend logic (how the page is built)

- Auditor pages are read-only and minimal; no action buttons.

Backend call and what the backend does

- Freezing a package emits `audit_package.frozen`, which triggers notification emails (see Part D).
- `verifier-cli` duplicates the Merkle logic so it works without our servers.

WHY we built it this way

- Auditors cannot be asked to trust the vendor; an offline verifier and read-only workspace give independent assurance.

C19. Controls, Compliance and Evidence Operations

Route: /controls, /compliance, /evidence-operations | Source: frontend/src/app/controls, compliance, evidence-operations

What the user sees

- Controls list with pass/fail and frameworks such as SOC2 and ISO27001.
- Evidence Operations (W24): list of evidence items with verify action.

Step-by-step flow (what you do, what happens)

1. Open Controls; results from `/control-evaluations`.
2. Click Evaluate on a test (`/control-tests/:id/evaluate`).
3. In Evidence Operations choose an item and **Verify** (`/evidence/:id/verify`).
4. Compliance posture on the Command Center updates.

Frontend logic (how the page is built)

- Posture percentage is computed from test results; failed controls link to remediation.

Backend call and what the backend does

- Control evaluations read evidence collected by connectors and the ledger; verification checks hashes.

WHY we built it this way

- Compliance must be continuous and evidence-based rather than a yearly spreadsheet.
- Linking controls to verifiable evidence removes disputes during audits.

C20. Risk & Exceptions

Route: /risk, /exceptions | Source: frontend/src/app/risk, exceptions

What the user sees

- Risk register (W27/28) and exception requests with expiry.

Step-by-step flow (what you do, what happens)

1. Record a risk, score it.
2. Request an exception to a control with a reason and an end date.
3. Approver reviews; the exception auto-expires.

Frontend logic (how the page is built)

- Forms with required reason and expiry; expiring items highlighted.

Backend call and what the backend does

- `exception.expired` is a Kafka event that triggers notification emails.

WHY we built it this way

- Exceptions are normal but dangerous when forgotten; mandatory expiry prevents permanent risk acceptance.

C21. Notification Center

Route: /notifications | Source: frontend/src/app/notifications/page.tsx

What the user sees

- A list of notifications with a status badge per delivery (for example DELIVERED, FAILED, DEAD_LETTERED).
- A warning banner when any delivery FAILED or DEAD_LETTERED.
- An Acknowledge button, and a link to the Email Template Studio.

Step-by-step flow (what you do, what happens)

1. Open the page; it calls `GET /api/v1/notifications` (filtered by your tenant and your principal id).
2. Read items; failed ones are highlighted.
3. Click **Acknowledge** (`POST /notifications/:id/acknowledge`); types SEEN / ACKNOWLEDGED / ACCEPTED / DECLINED exist in the backend.
4. Open Template Studio to see how emails look.

Frontend logic (how the page is built)

- If the backend errors or returns an empty list, the page shows **4 hard-coded SAMPLE_NOTIFICATIONS** and acknowledge only changes local state.
- Status badges map delivery states to colours.

Backend call and what the backend does

- `NotificationController` lists deliveries for `recipient_principal_id = user.id`.
- `NotificationDispatchService` chooses policy -> preference -> idempotent delivery row -> template -> channel (InApp, Email, Slack, Teams). Mandatory policies cannot be muted by preferences.
- Duplicate sends are suppressed with a unique key (Prisma P2002).

WHY we built it this way

- Security alerts are useless if they are ignored; delivery status and acknowledgement make 'who was told, and did they see it?' provable.
- Preferences exist so users control noise, but mandatory categories (legal, security) cannot be turned off.

Verified gaps on this page (see Part H)

- Sample fallback can make an empty real list look populated; it should be clearly labelled demo or removed.
- Kafka consumers use literal recipient ids such as 'tenant-admin', so real users may never see event-driven notifications (Part D).

C22. Email Template Studio

Route: /admin/email-templates | Source: frontend/src/app/admin/email-templates/page.tsx

What the user sees

- Left: **16 domain chips** + search + list of up to **226 templates** (ids like ZS-EML-IAM-005).
- Centre: preview with a **Desktop (600 px)** / **Mobile (380 px)** toggle and **Dark / Light** toggle, plus tabs HTML preview, Plain Text, Variables.
- Variables form: recipientFirstName, organizationName, referenceId, statusLabel, occurredAtLocal, timezone, actionUrl.
- Audit hash display, a recipient field with **Send Test**, a **Test Secret Gate** button and **Reset Variables**.

Step-by-step flow (what you do, what happens)

1. Pick a domain, then a template.
2. Edit variables; the preview re-renders via `POST /notifications/templates/:id/preview`.
3. Switch Desktop/Mobile and Dark/Light to check rendering; read the Plain Text tab (accessibility, spam filters).
4. Check the audit hash (SHA-256 of the render).
5. Enter a test recipient and click Send Test (`/templates/:id/send-test`).
6. Click Test Secret Gate to prove an email containing a bearer token is blocked.

Frontend logic (how the page is built)

- 1,080-line client page. If the template list from the server has fewer than 226 entries it **falls back to a locally generated list**.
- `generateClientPreview` renders locally if the server preview fails and runs its own regex secret check (bearer eyJ..., AKIA..., PRIVATE KEY, sk-...).
- The iframe preview uses sandboxed HTML so template code cannot touch the app.

Backend call and what the backend does

- Backend renders with `ProductionEmailTemplateEngine.render()`: fail-closed validation, escaped HTML, secret gate, render hash.
- Send-test renders, sends through the **mock** email channel, and writes a delivery audit row.

WHY we built it this way

- Emails are a legal and security surface (password resets, breach notices). A studio lets product, legal and security review every message before it ships.
- Dark/light and mobile previews catch broken layouts, which are common in email clients.
- The secret gate demo proves that credentials cannot leak by email, a typical audit question.

Verified gaps on this page (see Part H)

- **Send-test failure is swallowed:** the UI still says 'Dispatched successfully' with a mock delivery id; misleading.
- Backend send-test uses the mock channel, so no real email leaves the system.

C23. Platform Admin & Governance pages

Route: /admin, /operations and governance routes | Source: frontend/src/app/admin, operations

What the user sees

- **Platform Service Health (section 31/32):** readiness for each service.
- **G1 Launch Gate Protocol (0/8 GATE):** eight launch conditions with signed roster.
- **JIT Elevation Quorum (section 13, FOUR-EYES):** time-boxed privilege elevation.
- **GTM Pre-Flight Checklist (12/12 PASS):** go-to-market checks.
- **Operations & Entitlements (OPERATOR):** operator tools and feature entitlements.

Step-by-step flow (what you do, what happens)

1. Health: page loads `/observability/readiness` and shows green/red per service.
2. G1 gate: each approver signs (`/governance/g1-roster/sign`) until 8/8.
3. JIT: a user requests elevation (`/jit/elevate`), a second approver approves (`/jit/sessions/:id/approve`) and the requester completes step-up (`/jit/sessions/:id/step-up`).
4. Checklist: tick items to 12/12.
5. Operations: view and change entitlements.

Frontend logic (how the page is built)

- Strongly stateful pages: buttons appear only when the next step is valid.
- Badge counters are derived from the same state shown on the page.

Backend call and what the backend does

- shield-core governance and JIT modules enforce four-eyes and expiry and write audit events; observability reports readiness.

WHY we built it this way

- A security platform must be operated securely too: no standing admin rights (JIT), no single-person launch decision (G1), and visible service health.
- The four-eyes rule is the same pattern used for policies and SOAR, so teams learn one mental model.

C24. MSSP Fleet & 2-Party JIT

Route: /partner/fleet | Source: frontend/src/app/partner/fleet/page.tsx

What the user sees

- A fleet of customer tenants managed by a partner (MSSP) with a 2-PARTY badge.
- Request access to a customer and see approval state.

Step-by-step flow (what you do, what happens)

1. The MSSP analyst requests JIT access (`/partners/fleet/jit/request`).
2. The **customer** approves (`/partners/fleet/jit/approve-customer`).
3. Access is time-boxed and recorded on both sides.

Frontend logic (how the page is built)

- Shows pending, approved, expired lines with countdowns.

Backend call and what the backend does

- The partner module requires both the partner approver and the customer to approve; tokens expire; actions are attributed to the partner principal.

WHY we built it this way

- MSSPs need access to many customers but customers must stay in control; two-party approval satisfies both.

C25. G2 Experience Contracts

Route: /cases/incidents/command, shift-handover, /assets, /findings, risk executive, reports, export, /developer, /trust | Source: frontend/src/app/assets, findings, developer, trust, cases ...

What the user sees

- **Incident Command W19**, **SOC Shift Handover W20**, **Asset Inventory W29**, **Findings W30**, **Executive Risk W31**, **Audit Reports W32**, **Admin Export W36**, **Developer W37**, **Trust W38**.

Step-by-step flow (what you do, what happens)

1. Each page is a role-focused view: executives get risk summaries, engineers get API keys and docs, customers see the trust centre.
2. Reports can be generated and exported as sealed packages.

Frontend logic (how the page is built)

- 'Experience contracts' means each page promises a defined data shape to the backend-for-frontend (BFF) and displays contract health (HEALTHY_SYNCED) on the Command Center.

Backend call and what the backend does

- Served by shield-core BFF endpoints that aggregate cases, assets, findings and risk.

WHY we built it this way

- Different audiences need different views; defining contracts per view lets frontend and backend evolve independently with visible drift detection.

Part D

The email system in full

Email is where the platform talks to humans outside the browser: activation links, password resets, security notices, audit freezes. Because email can leak data or be abused, the platform treats it as a controlled, audited subsystem.

D1. Layers (from template to inbox)

Layer	File / class	What it does	Why
Template engine	`ProductionEmailTemplateEngine`	226 templates, render, validate, secret gate, render hash	One approved source of wording
Enum registry	`template-enums.registry.ts`	Allowed status/action/role labels	Stops raw internal codes and injection
Transactional service	`TransactionalEmailService`	Tenant isolation gate, https CTA gate, builds receipt with digest	Cross-tenant leaks are P0
Dispatch service	`NotificationDispatchService`	Policy -> preference -> idempotent row -> template -> channel	Controls who is told and avoids duplicates
Channels	InApp, Email, Slack, Teams	Deliver the rendered message	Different people read different places
Email channel	`EmailChannelService`	**MOCK**: logs a warning and returns delivered:true	Placeholder until a real provider is wired
Identity mailer	`identity-adapter/mail.service.ts MailService`	`sendOtp`, `sendOwnerInvitation` using the engine; nodemailer Gmail if EMAIL_USER and EMAIL_APP_PASSWORD set	Auth emails need a dedicated path
Event consumer	`DomainEventNotificationConsumer`	Kafka events -> notifications	Automatic notices when things happen

D2. Template catalogue (226 templates in 16 domains)

Ids follow `ZS-EML-<DOMAIN>-NNN` and each has a gate class: P0, P1, P2 or C (critical / compliance).

Domain	Count	Typical content
IAM	23	Account, passkey, password recovery (IAM-005 reset), sign-in alerts
ORG	13	Organisation, owner activation (ORG-002), membership
CONN	15	Connector connected, auth failed, sync issues
SEC	19	Security alerts and detections
ACT	13	Response action requested, approved, executed
ASSURE	21	Assurance and compliance notices
EVID	14	Evidence, audit packages frozen
AI	12	AI output blocked, governance
DEV	11	Developer and API key notices

Domain	Count	Typical content
BILL	18	Invoices, quota thresholds
SUP	10	Support
PRIV	11	Privacy requests
OFF	8	Offboarding
STAT	10	Status and maintenance
OPS	20	Internal operations (internal footer)
GOV	8	Governance
Total	**226**	

D3. The gates every email passes (fail closed)

Gate	Check	Result if it fails
Required fields	templateId, recipientFirstName, referenceId present	BadRequest; nothing rendered
Known template	templateId exists in the catalogue	BadRequest 'unknown template'
Allow-listed enums	status, action type, role labels come from the registry	Rejected
Secret gate	Body must not contain `bearer `, `private_key`, `BEGIN PRIVATE KEY`, `password=`, `api_key`, `client_secret`	Rejected before sending
Tenant isolation (Gate 3)	tenantId present; recipient contains '@'; resourceTenantId == tenantId	P0 log + BadRequest
CTA URL	Must be https (http only for localhost)	Rejected
HTML safety	All variables HTML-escaped	Prevents injection
Render hash	SHA-256 of {templateId, version v2.0, policyVersion iam-policy-1.0.0, plainTextBody, ctaUrl}	Stored as audit proof of exact content

Internal templates (ZS-EML-OPS-*) use an internal footer; the call-to-action URL has a fallback chain when no actionUrl is supplied.

WHY

- **Fail closed:** a wrong email to a wrong person is a data breach; refusing is safer than guessing.
- **Secret gate:** the most common email leak is pasting a token into text; the engine blocks the patterns automatically.
- **Render hash:** proves later exactly what was sent, which supports disputes and audits.
- **Plain text body:** required for accessibility and for mail filters that distrust HTML-only mail.

D4. Every email path and its real status

Email	Triggered by	Code path	Status
Owner activation (ORG-002)	Onboarding	`OnboardingService` -> `MailService.sendOwnerInvitation` -> engine	Rendered OK; **returns after engine path, real SMTP skipped**; link `/accept-invite` page missing
Password recovery OTP (IAM-005)	Forgot password	`/auth/password-recovery/request` -> `MailService.sendOtp` -> engine	OTP embedded in `password_reset_url`; **early return skips nodemailer and console fallback**
Tenant member invitation	Team invite	`CreateInvitation`	**No email sent**; token shown in UI
Template Studio send-test	Admin	`POST templates/:id/send-test` -> EmailChannelService (mock)	Writes delivery audit; **no real delivery**; UI shows success even on failure

Email	Triggered by	Code path	Status
Event notifications (audit frozen, exception expired, critical alert, dual custody request, quota, JIT request, incident declared, connector auth failed, AI blocked, maintenance, invoice, privacy request)	Kafka domain events	`DomainEventNotificationConsumer` -> dispatch service	Works end to end except **literal recipient ids** ('tenant-admin', 'soc-lead')
Generic transactional dispatch	Any caller	`TransactionalEmailService.dispatchTransactionalEmail`	Returns `DELIVERED` with a SHA-256 digest but **performs no transport**; no other callers yet

D5. Password recovery flow (step by step)

1. User enters email on the forgot-password screen; `POST /auth/password-recovery/request` (throttle 3 per minute).
2. Response is always generic ('if the account exists...'), so attackers cannot test emails.
3. Only principals with a LocalCredential get an OTP challenge (valid 10 minutes); the mail uses template ZS-EML-IAM-005.
4. User enters the code at `/verify`; the backend sets a short-lived recovery grant cookie.
5. User sets a new password at `/reset`: bcrypt hash stored, **all sessions revoked**.
6. User signs in again with the new password.

D6. How to test email safely today

1. Open /admin/email-templates and pick ZS-EML-IAM-005.
2. Fill variables and read the HTML and Plain Text tabs; note the audit hash.
3. Click Test Secret Gate and confirm it is blocked.
4. Send Test with your address; check backend logs for the mock channel warning (this proves the render and audit path, not real delivery).
5. To send real mail now, set `EMAIL_USER` and `EMAIL_APP_PASSWORD` (Gmail app password) in backend .env; the `MailService` nodemailer path is only reached if the engine path is bypassed - see Part H fix list.

Remaining email work (the 'email part remaining')

- Wire a real provider (SES, SendGrid, SMTP) behind `EmailChannelService` and make `TransactionalEmailService` actually send.
- Fix `MailService.sendOtp` / `sendOwnerInvitation` so a successful render is followed by real delivery, not an early return.
- Add the `/accept-invite` frontend page and send tenant member invitation emails.
- Resolve real recipient principal ids in `DomainEventNotificationConsumer`.
- Make Template Studio show the real send-test result (failure must not read as success).
- Remove or clearly label SAMPLE_NOTIFICATIONS in the Notification Center.

Part E

Backend services step by step

Each service is a NestJS application. All use `ValidationPipe({ whitelist: true, transform: true })`, which strips unknown fields and converts types. shield-core also uses cookie-parser and CORS configuration.

E1. Authentication and session lifecycle (shield-core)

1. Request hits `AuthController` with a throttle of 10 per minute.
2. `AuthService.login` finds the principal, rejects inactive users, loads the LocalCredential and compares with bcrypt.
3. Failures increment a counter; reaching `LOCKOUT_THRESHOLD` locks the account.
4. If email is not verified: 403 'Email not verified'.
5. `sessionContext.resolveBinding` verifies tenant membership.
6. Access JWT and refresh token are set as cookies; identity events are recorded.
7. Subsequent requests: guards decode the JWT, attach the principal and tenant, then RBAC permission guards run.

WHY

- bcrypt is a slow, salted hash that resists offline cracking.
- Lockout and throttling limit brute force; identical errors prevent account enumeration.
- Refresh tokens allow short-lived access tokens, shrinking the damage of a stolen token.

E2. Tenancy, RBAC and invitations

- Every table carries a tenant id; services filter on it and reject cross-tenant ids.
- Roles map to permissions such as `tenant:member:invite`; platform roles are separate from tenant roles and cannot be invited.
- Invitation: create (token 32 bytes, SHA-256 stored) -> accept (authenticated principal, email match, PENDING, not expired) -> TenantMembership with source INVITATION.

E3. Ingestion pipeline (shield-ingest)

1. Webhook request arrives (through the proxy which signs it).
2. `WebhookSignatureGuard` checks raw body, signature, timestamp window and replay nonce.
3. The connector id binds the request to a tenant.
4. Normalization converts the payload into the common event schema (protected baseline; not to be modified lightly).
5. Event is stored and published to Kafka; detections consume it.
6. Pollers (for example Azure Monitor) pull data on a schedule and follow the same path.

E4. Detection, alerts and cases

- Detection definitions are versioned; a match produces an alert with a reference to the exact version.

- Alert -> case copies references and writes an audit row in the same transaction.
- Domain events (for example `alert.critical`) feed notifications.

E5. AI services (shield-ai)

- `ModelArmorSafetyGatewayService` screens prompts and outputs.
- `GroundingGateGuard` requires answers to cite tenant evidence.
- AI can create proposals but cannot directly run live SOAR actions.
- `SafeDegradationService` swaps to a deterministic rule engine when the kill switch is on.

E6. SOAR and dual custody (shield-action)

- Risk classes R0-R4; R2, R3 and R4 need two people.
- Initiator must provide a FIDO2 signature; quorum TTL default 15 minutes.
- Statuses: PENDING_SECOND_SIGNATURE, QUORUM_REACHED, REJECTED, EXPIRED.
- Rollback uses a single-use token and a compensating plan.
- Executors: Cloud Armor, GCP IAM, Google Workspace.

E7. Evidence: Merkle ledger, KMS and verifier

- Audit records are hashed as leaves, combined into a tree (`ZS-MERKLE-V1`), and the root is anchored per epoch.
- Proofs let anyone verify one record belongs to a sealed epoch without seeing the others.
- Envelope encryption and crypto-shredding protect data at rest and support deletion rights.
- `verifier-cli` repeats the verification offline: exit 0 valid, non-zero tamper.

E8. Notification pipeline and Kafka

1. A service publishes a domain event using the outbox pattern (written with its database change).
2. `DomainEventNotificationConsumer` receives it (inbox dedupe prevents double processing).
3. `NotificationDispatchService` resolves policy and preferences and creates one delivery row per channel (idempotent).
4. The template engine renders; the channel delivers (InApp real; Email mock today).
5. The user acknowledges in the Notification Center; the acknowledgement is audited.

WHY

- Outbox/inbox means no event is lost if Kafka is briefly down and none is processed twice.
- Idempotent delivery rows make retries safe.

Part F

End-to-end ERB-01 flow

ERB-01 is the fixed demonstration story (stages 1-10 in the bottom bar). The table below maps each stage to the UI, the API and the owning service.

Stage	What happens	UI page	API / service
1	Sign in as analyst	/login	POST /auth/login - shield-core
2	Tenant and connector ready	/connectors	/connectors - shield-ingest
3	Telemetry arrives (signed webhook)	/ingestion	/ingestion/webhooks/:id - shield-ingest
4	Events normalized	/ingestion	normalization - shield-ingest
5	Detection rule executes	/detections	detections - shield-core / ingest
6	P1 alert generated	/alerts	/alerts
7	Case created	/cases	/alerts/:id/create-case
8	AI summary and hunting	/copilot, /hunting	shield-ai with grounding gate
9	Response proposal, dual approval, action	/response-proposals, /actions	shield-action quorum
10	Evidence sealed and verified	/ledger, /audit	shield-anchor, verifier-cli

Email touchpoints in this story: owner activation (before stage 1), critical alert notice (stage 6), dual-custody approval request (stage 9), audit package frozen (stage 10).

Part G

Setup and troubleshooting runbook

Practical steps to run the platform locally and fix the commonest errors.

G1. Start everything

1. Start PostgreSQL, Redis and Redpanda (docker compose).
2. In `backend/`: install, run migrations, start services (core 3001, ingest 3002, ai 3003, action 3004, anchor 3005).
3. In `frontend/`: `npm run dev` (port 3000).
4. Open http://localhost:3000.

G2. Fix 'Invalid credentials' on the login screen

Your screenshot shows email `analyst@zoikoshield-demo.com`, tenant `00000000-0000-4000-8000-000000000001`, banner 'CEDAR_POLICY_DENY' and 'Invalid credentials'. The banner is a fixed label; the real error is Invalid credentials: that user does not exist in your database or the password is wrong.

1. In `backend/` run: `npm run seed:platform-admin -- <email> <password>`.
2. Then: `npm run seed:dev-tenant -- --owner you@example.com --password 'YourPassword123!'` (refuses to run when NODE_ENV=production; default password `DevTenantOwner123!`).
3. The command prints the **tenant id** and owner credentials.
4. On /login enter that email, that password and that tenant id exactly.
5. Preset buttons (sarah.chen@acme.com etc., password `Shield@SecOps2026!`) work only if those users are seeded.

G3. Common errors

Symptom	Cause	Fix
Invalid credentials	User or tenant not seeded, wrong password	Run seed commands; use printed tenant id
403 Email not verified	Account not verified	Verify or seed with verified flag
Account locked	Too many failures (LOCKOUT_THRESHOLD)	Wait or reset in the database
502 Bad Gateway	A backend service is down or slow (15 s timeout)	Start the service; check SHIELD_*_URL
Empty lists, sample data	Backend error or empty and page shows samples	Check network tab and backend logs
Webhook 401	Bad signature, stale timestamp, replayed nonce	Use proxy signing; fresh nonce; clock within 300 s
400 on invite or onboarding	Payload does not match DTO	See Part H contract table

G4. Key environment variables

Variable	Purpose
SHIELD_CORE_URL / INGEST / AI / ACTION / ANCHOR	Where the proxy forwards

Variable	Purpose
BACKEND_PROXY_TIMEOUT_MS	Proxy timeout (default 15000)
WEBHOOK_HMAC_SECRET / WEBHOOK_HMAC_SECRETS	Webhook signing (global dev / per connector)
NEXT_PUBLIC_DEMO_FALLBACK	Opt-in demo data when 'true'
APP_BASE_URL	Base for email links (default http://localhost:3000)
EMAIL_USER / EMAIL_APP_PASSWORD	Gmail transport for MailService
LOCKOUT_THRESHOLD	Failed logins before lockout

Part H

Verified findings and remaining work

These are real differences between what the UI and backend expect, found by reading the code. None were silently changed in this task; they are listed so they can be fixed deliberately.

H1. Frontend / backend contract table

Area	Frontend sends / expects	Backend expects / returns	Status
Login	email, password, x-tenant-id	same	OK
Passkeys	options and verify endpoints	same	OK
Notifications list/ack	GET /notifications, POST acknowledge	same	OK
Onboarding	organizationName, slug, legalEntityName, environmentName, homeRegion	orderId, tenantName, tenantSlug, homeRegion, legalEntity{}, ownerEmail, accessDisclosureVersion	**Mismatch**
Invite member	invitedEmail, assignedRole	invitedEmail, roleId	**Mismatch (400)**
Invite response	Invitation object	{invitationId, expiresAt, token}	**Mismatch**
Accept invitation	reads inv.invitedEmail	returns TenantMembership	**Mismatch**
Owner activation page	/accept-invite expected by email link	endpoints /auth/owner-invitations/:token(/start)	**Page missing**

H2. Email gaps

- Email transport is a mock; `TransactionalEmailService` claims DELIVERED without sending.
- `MailService` OTP and owner invitation return early after the engine path (real send and console fallback skipped).
- Template Studio send-test failure displays as success; the template list needs 226 entries from the server or uses a local list.
- Kafka notification consumer uses literal recipient ids.
- Tenant member invitations send no email.

H3. Security and hardening gaps

- Dual-custody quorums are in an in-memory Map; need durable storage.
- Quorum secret key has a hard-coded default fallback; require a configured key.
- Static TrustBar values (5/5 OK, epoch, PQC seal) should be wired to real data.
- Login shows a fixed 'CEDAR_POLICY_DENY' label for any 401/403.
- Notification Center sample data should be labelled or removed.

H4. Documentation vs code

Topic	Older document says	Code says
Password hashing	Argon2id	bcrypt

Topic	Older document says	Code says
Ports	3000-3004	Also shield-anchor 3005
Demo fallback	Always 100% fallback	Only when NEXT_PUBLIC_DEMO_FALLBACK=true
Proxy on error	Returns demo data	Returns real 502 / error

H5. Recommended fix order

1. Align invite payload (`roleId`) and onboarding payload with the DTOs; surface errors in the UI.
2. Add the `/accept-invite` page.
3. Wire real email delivery and fix the early-return in `MailService`; make send-test truthful.
4. Resolve real recipient ids in the notification consumer.
5. Persist dual-custody quorums and remove the default secret.
6. Replace static TrustBar values and login policy label with real data.

End of guide. Re-run `python backend/scripts/generate-detailed-platform-guide-pdf.py` after code changes to refresh it.